NEUES GYMNASIUM
BOCHUM

Neues Gymnasium Bochum
- Europaschule in NRW -
Querenburger Straße 45
44789 Bochum

Implementation eines Finite-Elemente-Methode Simulation

Magnus Michl

4. März 2026

vorgelegt im Fach	Physik
im Schuljahr	2025/2026
Abgabetermin	4.03.2026
Betreuungslehrer	Herr Schostok

Inhaltsverzeichnis

1 Abstract **1**

2 Einleitung **1**

3 Mathematische Grundlagen **2**

3.1 Vorwissen 2

3.2 Analysis 3

3.2.1 Partielle Ableitung 3

3.2.2 Partielle Differenzialgleichung 5

3.2.3 Partielle Integration 5

3.2.4 Integration durch Substitution 6

3.2.5 Gauß-Quadratur 6

3.3 Linear Algebra 6

3.3.1 Matrix 7

4 Finite Elemente Methode **10**

4.1 Die Bedeutung von Differentialgleichungen für die Physik 10

4.2 Einführung in die Finite Elemente Methode 10

4.3 Schwache Formulierung 11

4.4 Ansatzfunktionen 12

4.4.1 Referenzelement 13

4.4.2 Koordinatentransformation 13

4.4.3 Lineare Ansatzfunktion 14

4.5 Ableitung der Ansatzfunktion 14
4.6 Diskretisierung 15
4.6.1 Zusammensetzung der globalen Matrix und Vektor 20
4.6.2 Betrachtung der Ansatzfunktionen 21
4.7 Randbedingungen 24
4.7.1 Dirichlet-Randbedingung 24
4.7.2 Neumann-Randbedingung 24
4.7.3 Robin-Randbedingungen 25
4.8 Anwendung des FEM Verfahrens 25
4.9 Poisson-Gleichung 26
4.10 Konvergenz in der FEM 28

5 Die Implementation 30

5.1 Programmiersprache 30
5.2 Python 30
5.3 Grundlagen der Programmierung 30
5.3.1 Kommentare 31
5.3.2 Variablen 31
5.3.3 Funktionen 32
5.3.4 Datentypen 33
5.3.5 Schleifen 34
5.3.6 OOP 35
5.3.7 Bibliotheken 36

5.3.8 Pip 38

5.3.9 Jupyter Notebooks 38

5.4 Poisson Gleichung 39

6 Fazit **40**

6.1 Mögliche weiterführende Themen und Bereiche 40

Quellenverzeichnis **42**

Weitere Quellen **46**

Nutzung von künstlicher Intelligenz **47**

Schlusserklärung **49**

1 Abstract

A multitude of important problems in various STEM fields such as Physics, Engineering et cetera are inevitably expressed as ordinary or partial differential equations (ODE / PDE). These equations are oftentimes too complex to be solved analytically and require a different approach, a numerical approach. One of these numerical methods is the so called finite-element-method (FEM), which approaches the problem by splitting the solution function into a finite number of element and computes the solution only at a finite number of points, hence the name. In this paper, the inner workings of the finite-element-method will be introduced and applied to a differential equation with real world use cases in the field of physics, the infamous poisson equation.

2 Einleitung

In der Physik wie auch vielen anderen wissenschaftlichen Feldern werden eine Unzahl an wichtigen Gleichungen durch sogenannte Differentialgleichungen (DGL) gegeben. Diese sind oft nicht analytisch lösbar. Die wesentliche Motivation dieser Facharbeit lässt sich aus der Tatsache herleiten, dass trotz der Existenz einer Gleichung in Form einer DGL oft keine formele Lösung vorliegt. Lösungen dieser DGL zu finden ist jedoch in vielen Felder von großer Bedeutung.

Diese Facharbeit beschäftigt sich mit der ***Finite-Elemente-Methode (FEM)***, ein numerisches Verfahren um Differentialgleichungen zu lösen. Dabei wird das Differentialgleichungs-Problem in ein Lineares Gleichungsystem umformuliert, welches effizient von Computern gelöst werden kann. Heutzutage findet die Finite-Elemente-Methode bereits in einer Vielzahl von MINT-Bereichen wie dem Ingenieurwesen Anwendung. Beispielsweise wird diese in Form der Finite-Elemente-Analyse (FEA) angewendet, um die strukturelle Integrität von Bauteilen unter Belastungen zu untersuchen. Ziel dieser Fachar-

beit ist die Vorgehensweise der Anwendung der Finite-Elemente-Methode zu illustrieren. Dabei für die Finite-Elemente-Methode beispielsweise an einer, in der Physik gebräuchlichen, Differentialgleichung angewendet.

3 Mathematische Grundlagen

3.1 Vorwissen

Dies sind mathematische Grundlagen die bereits im Schulunterricht besprochen wurden sind, diese werden als vorgegeben erwartet. Diese beinhalten unter anderem:

- Algebra
- Analysis
  - Ableitung
  - Formale Definition (H Methode)
  - Ableitungsregeln
    - Konstantenregel
    - Potenzregel
    - Faktorregel
    - Summenregel
    - Kettenregel
    - Produktregel
    - Natürliche Exponentialfunktion
    - Allgemeine Exponentialfunktion
    - Natürlicher Logarithmus
  - Integration

        - Hauptsatz der Differential- und Integralrechnung
        - Potenzregel
        - Integrieren anderer bekannter Funktionen
    - gewöhnliche Differentialgleichung erster Ordnung
        - homogene und inhomogene DGL
        - Lösungsansätze einer DGL
            - Seperation of Variables
            - taktisches "Raten"
- Linear Algebra
    - Vektoren und Skalare

Im Folgenden werden basierend auf den bestehenden Grundlagen weitere mathematische Konzepte eingeführt, welche für diese Facharbeit nötig sind.

3.2 Analysis

Die Analysis ist ein Teilgebiet der Mathematik. Neben der Differential- und Integralrechnung umfasst die Analysis weitere Gebiete, welche darauf aufbauen. Dazu gehören die Theorie der gewöhnlichen und partiellen Differentialgleichungen.(siehe wikipediaAnalysisWikipedia)

3.2.1 Partielle Ableitung

Bereits bekannt sollte das Konzept der Ableitung sowohl aus der Physik wie der Mathematik sein. Dabei besteht die Idee, das durch das Ableiten einer Funktion f eine Funktion f' entsteht, welche an einer beliebigen Stelle die Steigung der Funktion f beschreibt.

Dabei sind wir bekannt mit folgenden Schreibweisen:

$$f'(x) = \frac{\mathrm{d}f(x)}{\mathrm{d}x}$$

Diese folgten daraus, das eine beliebige (differenzierbare) Funktion f nur einen Parameter x besitzt, von dem der Funktionsterm abhängt.

$$f(x) = 10x^2$$

Und somit beschreibt die Ableitungsfunktion f' wie sich die Funktion f mit Respekt zu x verändert.
Nun können aber in vielen komplexeren Themengebiete folgende Funktionen auftreten, welche von mehr als einer Variable abhängig sind.

$$f(x, y) = 10xy - 10y$$

Dadurch gibt es mehrere Parameter welche den Funktionsterm beeinflussen und es ist nicht klar, welche Ableitungsfunktion mit f' gemeint ist.
Es wird die sogenannte ***partielle Ableitung*** eingeführt. Diese beschäftigt sich nur mit der partiellen (also teilweisen) Änderung der Funktion unter Beachtung eines einzelnen Parameters. Dies wird folgenderweise ausgeschrieben: Die partielle Ableitung der Funktion f nach x

$$\frac{\partial f}{\partial x} = 10y$$

Die partielle Ableitung der Funktion f nach y

$$\frac{\partial f}{\partial y} = 10x - 10$$

Dabei werden beim Ableiten alle Variablen, die nicht geändert werden, wie Konstanten behandelt.

3.2.2 Partielle Differenzialgleichung

Eine ***partielle Differentialgleichung (PDG)*** ist eine Gleichung, die die Beziehung zwischen einer Funktion mit zwei oder mehreren Argumenten und ihrer partiellen Ableitungen beschreibt. (siehe wikipediaPartielleDifferentialgleichung)
Fundamental erweitert die ***partielle Differentialgleichung*** gewöhnliche Differentialgleichungen um die Möglichkeit, die Änderung mehrerer Argumente in ein Verhältnis zueinander zusetzen. Beispielhaft könnte eine partielle Differentialgleichung folgendermaßen aussehen:

$$\frac{\partial f}{\partial x} - c\frac{\partial f}{\partial t} = 0$$

Im Kontext dieser Facharbeit werden unter anderem die Begriffe DGL und PDG austauschbar benutzt, da die FEM auf einer abstrakteren Ebene auf beide Arten von Differentialgleichungen anwendbar ist. Jedoch wird bei mit FEM lösbaren Differentzialgleichungen meistens über PDG gesprochen, da diese in der Physik durch viele Faktoren wie mehrere Raumdimensionen leicht auftreten können.

3.2.3 Partielle Integration

Die partielle Integration (teilweise Integration, Integration durch Teile, lat. integratio per partes), auch Produktintegration genannt, ist in der Integralrechnung eine Möglichkeit zur Berechnung bestimmter Integrale und zur Bestimmung von Stammfunktionen. Sie bildet das Gegenstück zur Produktregel der Differentialrechnung. (siehe wikipediaPartielleIntegration)

$$\int_a^b f'(x)g(x)\,dx = \big[f(x)g(x)\big]_a^b - \int_a^b f(x)g'(x)\,dx$$

3.2.4 Integration durch Substitution

Die Integration durch Substitution oder die Substitutionsregel ist eine wichtige Methode in der Integralrechnung, um Stammfunktionen zu finden und bestimmte Integrale auszuwerten. Die Substitutionsmethode erlaubt es, einen „komplizierten“ Integranden durch einen „einfachen“ Integranden zu ersetzen und damit das gegebene Integral auf ein einfacher handhabbares Integral zurückzuführen. (siehe wikipediaIntegrationSubstitution)

$$\int_a^b f(\varphi(x))\frac{\mathrm{d}\varphi(x)}{\mathrm{d}x}\,dx = \int_{\varphi(a)}^{\varphi(b)} f(u)\,du$$

3.2.5 Gauß-Quadratur

Die Gauß-Quadratur ist ein Verfahren zur numerischen Integration, das bei gegebenen Freiheitsgraden eine optimale Approximation des Integrals liefert.(siehe wikipediaGauQuadraturWikipedia)

$$\int_{-1}^{1} F(\xi)\,d\xi = \sum_i F(\xi_i)w_i \tag{1}$$

Wobei $F(x)$ einem Polynom entspricht.

3.3 Linear Algebra

Die lineare Algebra (auch Vektoralgebra) ist ein Teilgebiet der Mathematik, das sich mit Vektorräumen beschäftigt. Die lineare Algebra entstand aus zwei konkreten Anforderungen heraus: einerseits dem Lösen von linearen Gleichungssystemen, andererseits der rechnerischen Beschreibung geometrischer Objekte (vgl. wikipediaLineareAlgebra)
Im Unterricht wuren zu diesem Thema bereits im kurzen ***Vektoren*** und ***Skalare*** behandelt. Dieser Bereicht wird hier um das mathematische Object der ***Matrix*** erweitert.

3.3.1 Matrix

In der Mathematik versteht man unter einer ***Matrix*** (Plural ***Matrizen***) eine rechteckig angeordnete Tabelle von sogenannten Elementen, diese Elemente sind für unsere Zwecke meist reele Zahlen (vgl. wikipediaMatrixMathematik). Matrizen können sowohl mit anderen Matrizen multipliziert werden, als auch mit Vektoren, wobei bei letzterem ein neuer Vektor ensteht. Eine Matrix wird primär durch ihre rechteckige Anordnung von Elementen in m Zeilen und n Spalten charakterisiert. Eine Solche Matrix beschreibt man auch als $m{\times}n$ ***Matrix***.

$$M = \begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{bmatrix}$$

Multiplikation Eine $m \times n$ Matrix kann mit einer nxp Matrizen multipliziert werden, woraus eine $m \times p$ Matrix entsteht (vgl. wikipediaMatrizenmultiplikationWikipedia). Oder mit Vektoren (n Komponenete) multipliziert werden, wobei das Resultat ein Vektor mit m Komponenten sein wird. Dabei wird bei Matrix-Vektor-Multiplikation zwischen „row-first- und „column-first"Multiplikationen unterschieden, für diese Facharbeit werden wir uns jedoch nur mit „column-firstMultiplikationen beschäftigen. Diese sind daran erkennbar zu machen, dass der Vektor jeweils Rechts von der Matrix steht:

$$\begin{bmatrix} a & b & c \\ d & e & f \end{bmatrix} \begin{pmatrix} x \\ y \\ z \end{pmatrix} = \begin{pmatrix} ax + by + cz \\ dx + ey + fz \end{pmatrix}$$

Multpikation einer 2×3 Matrix mit einem 3d Vektor resultiert in einem 2d Vektor

Zudem ist zu beachten das bei der Multiplikation von Matrizen nicht das Kommuntativgesetz gilt.

$$AB \neq BA$$

Für weitere Informationen, diese Quelle geht genauer in genannte Aspekte ein: https://mbernste.github.io/p

Einheitsmatrix Die Einheitsmatrix oder Identitätsmatrix ist in der Mathematik eine quadratische Matrix, deren Elemente auf der Hauptdiagonale eins und überall sonst null sind. (siehe wiki:Einheitsmatrix 2026) Eine Besonderheit der Einheitsmatrix ist, dass eine Multiplikation mit dieser Matrix sowohl mit einem passenden Vektor als auch Matrix keine Transformation durchführt:

$$E\vec{x} = \vec{x}$$

$$EM = M$$

So ist Beispielsweise eine 3×3 Einheitsmatrix definiert als:

$$E = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

Einsvektor Dieser Begriff wird im Rahmen dieser Facharbeit eingeführt, hiermit ist ein Vektor gemeint, dessen Komponente alle 1 sind:

$$\mathbb{1} = \begin{pmatrix} 1 \\ \vdots \\ 1 \end{pmatrix}$$

Wichtig ist, dass dies niemals mit einem ***Einheitsvektor*** verwechselt werden sollte. Dies ist ein Vektor, dessen Länge gleich eins ist: $|\vec{v}| = 1$.

Division von Matrizen Eine weitere bekannte mathematische Operation neben der Multiplikation ist die Division. Die Division einer Matrix mit einer Anderen ist nicht formal definiert, wie es bei beispielsweise Skalaren der Fall wäre. Stattdessen multipliziert man mit der Inverse einer Matrix, wenn man durch diese dividieren will. Die ***Inverse Matrix*** ist so definiert, dass deren Produkt mit der originalen Matrix eine Einheitsmatrix ergibt.

$$MM^{-1} = M^{-1}M = E$$

Lineare Gleichungssysteme Eine weitere Anwendung für Matrizen liegt beim Lösen von Linearen Gleichungsystem. So kann ein Lineares Gleichungssystem:

$$\begin{cases} 2x + 3y - z = 1 \\ 4x + y + 3z = 18 \\ x - 2y - z = -4 \end{cases}$$

Umgeschrieben werden in Matrix-Form:

$$\begin{bmatrix} 2 & 3 & -1 \\ 4 & 1 & 3 \\ 1 & -1 & -1 \end{bmatrix} \begin{pmatrix} x \\ y \\ z \end{pmatrix} = \begin{pmatrix} 1 \\ 18 \\ -4 \end{pmatrix}$$

Der Vorteil dieser Schreibweise liegt darin, dass das Gleichungssystem nun gelöst werden können indem beide Seiten mit dem Inverse der Matrix multiplizieren:

4 Finite Elemente Methode

Im Wesentlichen ist die Finite Elemente Methode eine Methode um numerisch Differentialgleichungen wie partielle Differentialgleichung zu lösen. Sie erlaubt es DGL / PDG in Probleme linearer Gleichungsysteme umzuformulieren, welche effizient von Computern gelöst werden können. Dies ist von großem Nutzen für die Physik, da für viele DGLs / PDGs oft keine analytische Lösung existiert oder oft zu komplex ist, um in echten Anwendungsbereichen von Menschen ausgerechnet zu werden.

4.1 Die Bedeutung von Differentialgleichungen für die Physik

Die Motivation Differential Gleichung zu lösen liegt in ihrer Bedeutung für die Phyisk.

Differentialgleichungen sind in der Physik allgegenwärtig — die Naturgesetze sind Differentialgleichungen. Die fundamentalen Gleichungen der Physik, die Bewegungsgleichungen der Newtonschen und der relativistischen Mechanik, die Maxwell-Gleichungen des Elektromagnetismus, die Schrödingergleichung in der Quantenmechanik, die Einsteingleichungen in der allgemeinen Relativitätstheorie etc. etc. — Differentialgleichungen,Differentialgleichungen, und . . . Differentialgleichungen.(siehe Heinzle, S. 5)

4.2 Einführung in die Finite Elemente Methode

Ein fundamentales Problem beim Lösen von Differential Gleichung mit Hilfe von Computern liegt in ihrer Eigenschaft der Kontinuität. Computer sind grundlegend in ihrem Aufbau für diskrete Rechnungen ausgelegt und scheinen somit nicht für das Lösen von DGL's geeignet zu sein. Dies umgeht die FEM indem man die Kontinuerliche Lösung einer DGL in eine Diskretere Form umwandelt. Anstelle alle Lösungen einer DGL in einem kontinuerlichen Interval zu finden, untersucht man nur eine diskrete Menge an Werten für die man die DGL löst. Diese kann in Form eines Gleichungssystems, oft in Matrix-

Schreibweise, formuliert werden. Dies ist von Vorteil, da gewiese Hardware-Componente (wie GPU's oder TPU's) auf Linear Algebra Operationen wie Matrix- / Vektorrechnungen optimisiert sind. Um am Ende wieder auf eine kontinuerliche Lösung zu kommen, verwendet man sogenannte ***Ansatzfunktionen***, welche erlauben zwischen den diskreten Werten zu interpolieren. Diese schauen wir uns zu einem später Zeitpunkt nochmals genauer an.

4.3 Schwache Formulierung

Die meisten maßgeblichen Gleichungen physischer und technischer Systeme können nicht mit der Finite-Elemente-Methode gelöst werden, ohne sie vorher auf ihre schwache Formulierung zu reduzieren.(vgl. Oluwole 2011) Viele in der Physik vorkommende DGL stellen hohe Anforderungen an die Lösungen, welche oft nicht von der FEM erreicht werden können. Um diese Forderungen zu umgehen, wird die DGL erst in eine ***Schwache Form*** (engl. ***Weak Form***) umgeschrieben, welche „schwächere" Anforderung darstellt und somit mit Hilfe der FEM gelöst werden kann.
„In einer schwachen Formulierung müssen Gleichungen oder Bedingungen nicht mehr absolut (im klassischen Sinne) erfüllt sein (was oft nicht einmal wohldefiniert ist), sondern besitzen nur noch schwache Lösungen in Bezug auf bestimmte ‚Testvektoren' oder ‚Testfunktionen'." (vgl. wikipediaWeakFormulation)

$$u''(t) = f(t)$$

Um eine DGL in ihre ***Schwache Form*** umzustellen, multipliziert man die Terme auf beiden Seiten mit einer ***Testfunktion*** $v(t)$ multipliziert, dabei muss folgende Gleichung für alle Testfunktionen $v(t)$ gültig sein:

$$u''(t)v(t) = f(t)v(t) \tag{2}$$

Und anschließend wird auf beiden Seiten das Integral im gesuchten Interval gezogen. Für dieses Beispiel gehen wir vom Interval $[0; L]$ aus:

$$\int_0^L u''(t)v(t)dt = \int_0^L f(t)v(t)dt \tag{3}$$

Nun kann partielle Integration angewendet werden:

$$\int_0^L u''(t)v(t)\,dt = \left[u'(t)v(t)\right]_0^L - \int_0^L u'v'(t)\,dt = \int_0^L f(t)v(t)\,dt \tag{4}$$

4.4 Ansatzfunktionen

Die Finite-Elemente-Methode gibt durch ihre Methodik die genaue Lösung der Differentialgleichung oder ihrer Schwachen Form nur für eine diskrete Menge an Knoten an. Der Funktionswert für die Elemente zwischen diesen Knoten sind nicht gegeben und werden durch die sogenannten ***Ansatzfunktionen*** berechnet. Diese orientieren sich dabei an den Werten der Knoten des jeweiligen Elementes. In Rahmen dieser Facharbeit betrachten wir exklusiv Ansatzfunktionen von Elementen im 1-dimensionalen Fall, hierbei sind für jedes Element jeweils 2 Knoten gegeben. Dabei ergibt sich der Funktionwert des Elementes durch:

$$u^e(x) = N_0^e(x)u_0^e + N_1^e(x)u_1^e \tag{5}$$

Wobei u_0^e der Wert des ersten (linken) Knoten des Elements und u_1^e den Funktionswert des zweiten (rechten) Knoten darstellt. So geben N_0^e und N_1^e an, wie der Funktionswert des Elements aus den Werten der Knoten zusammengestellt wird. Dabei muss sicher gestellt werden das die Wertemenge von N_0^e und N_1^e im Interval $[0; 1]$ liegen.

4.4.1 Referenzelement

Da in den meisten Fällen die Ansatzfunktionen aller Elemente derselben Art[1] entsprechend, ist es sinnvoll, ein ***Referenzelement*** zu definieren. Dieses ***Referenzelement*** ist in einem anderen Koordinatenraum definiert und ermöglicht den jeweiligen Elementen durch Koordinatentransformation dessen Ansatzfunktion zu benutzen. Dadurch können die elementweise definierten Ansatzfunktionen N_0^e und N_1^e, welche im Interval $[0; L]$ definiert waren, durch Referenz-Ansatzfunktion N_0 und N_1 beschrieben werden. Dabei ist zu beachten das N_0 und N_1 für das Interval $[-1; 1]$ definiert sind.

4.4.2 Koordinatentransformation

Das Referenzelement hat einen Definitionsbereich $D = [-1; 1]$. Wenn wir Werte x des Koordinatenraum eines Element e, mit Definitionsbereich $D_e = [a; b]$ wobei $D \in [0; L]$, in den Koordinatenraum ξ des Refenrelement umrechnen wollen, gilt folgende Gleichung:

$$x = \phi_e(\xi) \qquad \text{und} \qquad \xi = \phi_e^{-1}(x)$$

Unter Annahme das ϕ_e linear ist.

$$\phi_e(\xi) = \frac{1}{2}(a+b) + \frac{1}{2}(b-a)\xi \tag{6}$$

$$\phi_e^{-1}(x) = \frac{\frac{1}{2}(a+b) - x}{\frac{1}{2}(a-b)} \tag{7}$$

Dabei ergibt sich für die elementweise definierten Ansatzfunktionen N_i^e:

$$N_i^e(x) = N_i^e(\phi_e(\xi)) = N_i(\xi) \tag{8}$$

[1] derselben Art sagt in diesem Kontext aus, dass beispielsweise alle Ansatzfunktionen linear verlaufen und die Werte der Knote auf die selbe Weise zusammen stellen

Dabei ist zu bemerken das N_i die Ansatzfunktion des Referenzelement im anderen Koordinatenraum ist.

4.4.3 Lineare Ansatzfunktion

Für den Rahmen der Facharbeit betrachten wir nur die Anwendung linearer Ansatzfunktionen. Diese sind für ein 1-dimensionales Referenzelment gegeben durch:

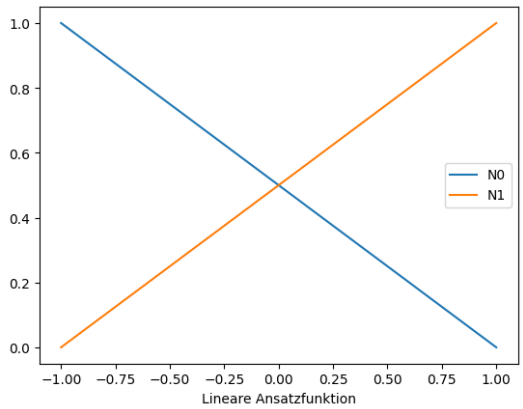

Abbildung 1: Lineare Ansatzfunktionen $N_0(\xi)$ und $N_1(\xi)$ des Referenzelementes für $\xi \in [-1; 1]$.

$$N_0(\xi) = \frac{1}{2} - \frac{1}{2}\xi \tag{9}$$

$$N_1(\xi) = \frac{1}{2} + \frac{1}{2}\xi \tag{10}$$

4.5 Ableitung der Ansatzfunktion

Die Ableitung einer linearen Ansatzfunktion für das Referenzelement ist recht simpel:

$$\frac{\mathrm{d}N_0(\xi)}{\mathrm{d}\xi} = -\frac{1}{2} \tag{11}$$

$$\frac{\mathrm{d}N_1(\xi)}{\mathrm{d}\xi} = \frac{1}{2} \tag{12}$$

Jedoch ist zu beachten das die Ableitung der Ansatzfunktion des Referenzelementes aufgrund der Koordinatentransformation nicht gleich der Ableitung der Ansatzfunktion

der einzelnen Elemente ist.

$$\frac{\mathrm{d}N_i^e(x)}{\mathrm{d}x} = \frac{\mathrm{d}N_i^e(\phi_e(\xi))}{\mathrm{d}x} = \frac{\mathrm{d}N_i(\xi)}{\mathrm{d}x} \tag{13}$$

Nun kann die Kettenregeln angewendet werden.

$$\frac{\mathrm{d}N_i(\xi)}{\mathrm{d}x} = \frac{\mathrm{d}N_i(\xi)}{\mathrm{d}\xi}\frac{\mathrm{d}\xi}{\mathrm{d}x} \tag{14}$$

Daraufhin kann x durch $\phi_e(\xi)$ ersetzt werden, um nicht mit zwei Koordinatenräumen zu kämpfen. Diese Schreibweise wird noch zu einem späteren Zeitpunkt von Nutzen sein.

$$\frac{\mathrm{d}N_i(\xi)}{\mathrm{d}\xi}\frac{\mathrm{d}\xi}{\mathrm{d}x} = \frac{\mathrm{d}N_i(\xi)}{\mathrm{d}\xi}\left(\frac{\mathrm{d}x}{\mathrm{d}\xi}\right)^{-1} = \frac{\mathrm{d}N_i(\xi)}{\mathrm{d}\xi}\left(\frac{\mathrm{d}phi_e(\xi)}{\mathrm{d}\xi}\right)^{-1} \tag{15}$$

4.6 Diskretisierung

Die ***Diskretisierung*** ist ein Grundbegriff aus der Mathematik und bezeichnet alle Methoden, die aus einem stetigen Problem ein diskretes Problem machen. (siehe wikipedia-DiskretisierungWikipedia) Dies wird im Kontext der vorherig genannten DGL angewandt. Eine kontinuierliche (stetige) Lösungsfunktion u_{stetig} im gesuchten Intervall $[0; L]$ zu finden, würde bedeuten für jeden Wert $x \in [0; L]$ ihren entsprechenden Funktionswert $u_{stetig}(x)$ zu finden. Dies bedarf einen genauen Funktionsterm, welcher analytisch ermittel werden müsste.
Stattdessen befasst man sich *nur* mit einer diskreten Menge x_i an Werten für welche die entsprechenden Funktionswerte u_i ermittelt werden. Die Gesamtheit der Punkte, in denen man Werte vorgibt, nennt man ***Knoten*** (engl. Nodes) (siehe wiasberlin).

Formulierung als lineares Gleichungssystem Das Ziel der Diskretisierung ist es, aus der Schwachen Form letzendlich ein lineares Gleichungsystem aufzustellen, wel-

ches die Lösung der Schwachen Form für eine diskrete Menge an Werten ausrechnet, diese Werte bezeichnet man als ***Knoten***. Mit diesen Knoten kann die Lösungsfunktion u als eine Linearkombination der diskreten Funktionswerte u_i und sogenannten ***Hütchenfunktionen*** ϕ_i dargestellt werden:

$$u^n(x) = \sum_{i=0}^{n-1} u_i \phi_i(x) \tag{16}$$

Dabei stellt n die Anzahl an Knoten dar, u_i den diskreten Funktionswert an Index i und ϕ_i die Hütchenfunktion an Index i. Man merke an, dass wir die Summanten von Index 0 bis $n-1$ summieren, da mit nullbasierter Indizierung gearbeitet wird. Die Hütchenfunktion ist dabei vorest als eine Funktion zu verstehen, welche an einem diskreten Wert u_i den Funktionswert 1 annimmt und für alle anderen diskreten Werte den Funktionswert 0. So ist eine lineare Hütchenfunktion definiert als:

$$\phi_i(x) = \begin{cases} \frac{x-x_i}{x_i-x_{i-1}} & x \in [x_{i-1}, x_i] \\ \frac{x_{i+1}-x}{x_{i+1}-x_i} & x \in [x_i, x_{i+1}] \\ 0 & \text{sonst} \end{cases} \tag{17}$$

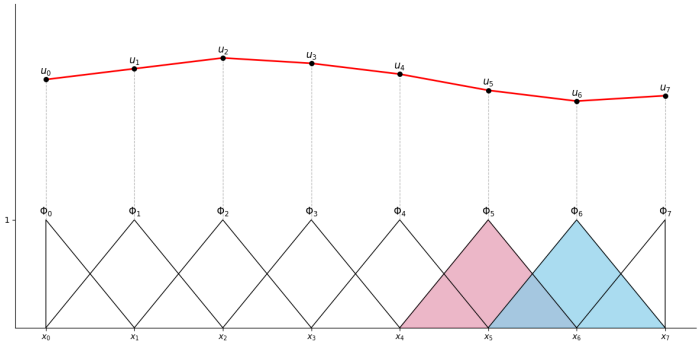

Abbildung 2: Eine Funktion linear interpolierter diskreter Funktionswerte. Zeigt die Korrelation zwischen diskreten Funktionswerten u_i und der Entsprechenden Hütchenfunktionϕ_i

Dabei sind x_i die Stellen für welche die diskreten Funktionswerte u_i gegeben sind: $u(x_i) = u_i$. Durch diese Definition nimmt die Funktion u^n an Stellen x_i den entsprechenden Funktionswert u_i an. In Abbildung 2 wird illustriert, wie eine Funktion u an Stellen x_i den Funktionswert u_i annimmt. Zwischen den diskreten Funktionswerten ist die Funktion linear interpoliert, verantwortlich dafür sind die gegebenden linearen Hütchenfunktionen ϕ_i.

Galerkin-Methode Die Galerkin-Methode ist ein numerisches Verfahren zur näherungsweisen Lösung von Operatorgleichungen, wie beispielsweise partiellen Differentialgleichungen (vgl. wikipediaGalerkinMethodeWikipedia). Die Galerkin-Methode legt nahe, dass die Testfunktion v mit der Hütchenfunktion ϕ übereinstimmt (vgl. Yaniv):

$$v(x) = \phi(x) = \sum_{i=0}^{n-1} \phi_i(x) \tag{18}$$

Nun setzen wir die Terme u^n und v in die Schwach Form ein und erhalten somit:

$$\begin{aligned} &\Big[\underbrace{\left(\sum_{i=0}^{n-1} \phi_i(x)\right)}_{v(x)} \underbrace{\left(\sum_{j=0}^{n-1} \frac{\mathrm{d}}{\mathrm{d}x} u_j \phi_j(x)\right)}_{u'(x)}\Big]_0^L \\ &- \int_0^L \underbrace{\left(\sum_{j=0}^{n-1} \frac{\mathrm{d}}{\mathrm{d}x} u_j \phi_j(x)\right)}_{u'(x)} \underbrace{\left(\sum_{i=0}^{n-1} \frac{\mathrm{d}}{\mathrm{d}x} \phi_i(x)\right)}_{v'(x)} dx = \int_0^L f(x) \underbrace{\left(\sum_{i=0}^{n-1} \phi_i(x)\right)}_{v(x)} dx \end{aligned} \tag{19}$$

Dabei ist zu beachten, dass die Ableitung bereits in die Summe geschrieben wurde, da hier die Summenregel gilt:

$$\frac{\mathrm{d}}{\mathrm{d}x} \sum f(x) = \sum \frac{\mathrm{d}}{\mathrm{d}x} f(x)$$

Da u_j eine Konstante ist und wir die Multiplikation zweier Summen weiter vereinfachen

können, lässt sich die Formel kürzen:

$$[\sum_{i=0}^{n-1}\sum_{j=0}^{n-1} u_j\phi_i(x)\frac{\mathrm{d}\phi_j(x)}{\mathrm{d}x}]_0^L$$
$$-\int_0^L \left(\sum_{i=0}^{n-1}\sum_{j=0}^{n-1} u_j\frac{\mathrm{d}\phi_j(x)}{\mathrm{d}x}\frac{\mathrm{d}\phi_i(x)}{\mathrm{d}x}\right) dx = \int_0^L f(x)\left(\sum_{i=0}^{n-1}\phi_i(x)\right) \tag{20}$$

In den nächsten Schritten wird die Ableitung $\frac{\mathrm{d}}{\mathrm{d}x}f$ auf f' aus Gründen der Lesbarkeit gekürzt.

$$[\sum_{i=0}^{n-1}\sum_{j=0}^{n-1} u_j\phi_i(x)\phi_j'(x)]_0^L - \int_0^L \left(\sum_{i=0}^{n-1}\sum_{j=0}^{n-1} u_j\phi_j'(x)\phi_i'(x)\right) dx = \int_0^L f(x)\left(\sum_{i=0}^{n-1}\phi_i(x)\right) \tag{21}$$

Linear Algebra Jedoch ist die Formel selbst in dieser kompakteren Schreibweise recht komplex. Im nächsten Schritt wird diese Formel durch Vektor- und Matrixschreibweise so umgeformt, dass wir eine vergleichsweise einfache Formel $A\vec{u} = \vec{F}$ erhalten.
So definieren wir zuerst eine Menge an Vektoren:

$$\vec{\phi}' = \begin{pmatrix} \phi_0' \\ \vdots \\ \phi_{n-1}' \end{pmatrix} \qquad \vec{\phi} = \begin{pmatrix} \phi_0 \\ \vdots \\ \phi_{n-1} \end{pmatrix} \qquad \vec{u} = \begin{pmatrix} u_0 \\ \vdots \\ u_{n-1} \end{pmatrix}$$

Diese lassen uns Gleichung (16), (18) und ihre jeweiligen Ableitungen folgenderweise umschreiben:

$$u^n(x) = \sum_{i=0}^{n-1} u_i\phi_i(x) = \vec{\phi}^T(x)\vec{u} \tag{22}$$

$$u^{n\prime}(x) = \sum_{i=0}^{n-1} u_i\phi_i'(x) = \vec{\phi}'^T(x)\vec{u} \tag{23}$$

$$v(x) = \phi(x) = \sum_{i=0}^{n-1} \phi_i(x) = \mathbb{1}^T\vec{\phi}(x) \tag{24}$$

$$v'(x) = \phi'(x) = \sum_{i=0}^{n-1} \phi_i'(x) = \mathbb{1}^T \vec{\phi}'(x) \tag{25}$$

Dabei ist zu beachten, dass ein Vektor mit n Komponenten in diesem Fall wie eine $n \times 1$ Matrix behandelt wird. Deshalb müssen die Vektoren auf der linken Seite transponiert werden, da eine Matrix Multiplikation zwischen zwei $n \times 1$ Matrix nicht erlaubt ist. Das Transponieren wandelt dies in eine erlaubte Operation zwischen einer $1xn$ Matrix und $nx1$ Matrix um, das Ergebnis ist eine $1x1$ Matrix, ein Skalar. Im Folgenden wird diese neue Schreibweise erneut in die Schwache Form eingesetzt:

$$[\underbrace{\vec{\phi}^T(x)\vec{u}}_{u'(x)} \underbrace{\mathbb{1}^T\vec{\phi}(x)}_{v(x)}]_0^L - \int_0^L \underbrace{\mathbb{1}^T\vec{\phi}(x)}_{v'(x)} \underbrace{\vec{\phi}^T(x)\vec{u}}_{u'(x)}\, dx = \int_0^L f(x) \underbrace{\mathbb{1}^T\vec{\phi}(x)}_{v(x)}\, dx \tag{26}$$

Aus Gründen der Simplizität nehmen wir im weiteren für die Randbedingungen an: $u'(0) = 0$ und $u'(L) = 0$. Damit lässt sich die Gleichung weiter vereinfachen:

$$-\int_0^L \underbrace{\mathbb{1}^T\vec{\phi}(x)}_{v'(x)} \underbrace{\vec{\phi}^T(x)\vec{u}}_{u'(x)}\, dx = \int_0^L f(x) \underbrace{\mathbb{1}^T\vec{\phi}(x)}_{v(x)}\, dx \tag{27}$$

$$-\mathbb{1}^T \int_0^L \vec{\phi}(x)\vec{\phi}^T(x)\vec{u}\, dx = \mathbb{1}^T \int_0^L f(x)\vec{\phi(x)}\, dx \tag{28}$$

Es sei angemerkt, dass der Vektor $\mathbb{1}$ ein Result der Annahme $v(x) = \phi(x)$ sei. Im Fall einer beliebigen Testfunktion $v(x)$ würde dieser Vektor durch Vektor $\vec{V}$ ersetzt werden, welcher diskrete Werte der Testfunktion enthält. Da dieser jedoch jedoch für alle Testfunktion $v(x)$ gelten muss, ist die Lösung deckungsgleich und der Vektor lässt sich ebenfalls von beiden Seiten kürzen. Hinzu sei angemerkt, dass die Multiplikation von ϕ (einer $n \times 1$ Matrix) und ϕ^T (einer $1 \times n$ Matrix) eine weitere Matrix $n \times n$ ergibt. Diese nennen wir A.

$$\underbrace{\left(\int_0^L \vec{\phi}(x)\vec{\phi}^T(x)\, dx\right)}_{A} \vec{u} = \underbrace{\int_0^L -f(x)\vec{\phi(x)}\, dx}_{\vec{F}} \tag{29}$$

Somit haben wir endlich die finalle Form erreicht:

$$A\vec{u} = \vec{F} \tag{30}$$

Diese lässt sich leicht lösen.

$$\vec{u} = A^{-1}\vec{F} \tag{31}$$

4.6.1 Zusammensetzung der globalen Matrix und Vektor

Unter Betrachtung der oben genannten Hütchenfunktionen kann die Matrix A folgenderweise berechnet werden:

$$A_{ij} = \int_0^L \phi_i'(x)\phi_j'(x)\,dx \tag{32}$$

Dabei ist zu beachten, dass eine entsprechende Hütchenfunktion ϕ_i und ihre Ableitung ϕ_i' nur in einem gewissen Interval $[x_{i-1}; x_{i+1}]$ größer null sind. Dies bedeutet das für zwei Hütchenfunktionen ϕ_i und ϕ_j mit Intervallen $[x_{i-1}; x_{i+1}] \cap [x_{j-1}; x_{j+1}] = \emptyset$, sowohl das Produkt der Funktionen wie ihrer Ableitung gleich null ist.

$$A_{ij} = 0 \quad \forall |i-j| > 1 \tag{33}$$

Beispielsweise wäre die Matrix A für eine Lösungsfunktion u mit Definitionsbereich $[0; 1]$ und 3 Elementen mit gleicher Länge(4 Knoten), folgenderweise definiert.

$$\phi_i'(x) = \begin{cases} \frac{1}{x_i - x_{i-1}} & x \in [x_{i-1}; x_i] \\ \frac{1}{x_{i+1} - x_i} & x \in [x_i; x_{i+1}] \\ 0 & \text{sonst} \end{cases} \tag{34}$$

Mit der Annahme, alle Elemente haben die selbe Länge: $|x_i - x_{i-1}| = |x_{i+1} - x_i| = \frac{1-0}{3}$.

$$\phi_i'(x) = \begin{cases} 3 & x \in [x_{i-1}; x_i] \\ -3 & x \in [x_i; x_{i+1}] \\ 0 & \text{sonst} \end{cases} \tag{35}$$

$$A = \begin{bmatrix} 3 & -3 & 0 & 0 \\ -3 & 6 & -3 & 0 \\ 0 & -3 & 6 & -3 \\ 0 & 0 & -3 & 3 \end{bmatrix} \tag{36}$$

4.6.2 Betrachtung der Ansatzfunktionen

Wie in Abschnitt 4.4 jedoch erläutert wird, benutzen wir in der Praxis keine global definierten Hütchenfunktionen sondern für einzelne Elemente definierte Ansatzfunktionen. Diese können wiederrum durch die Ansatzfunktionen des Referenzelementes ersätzt werden, jedoch wird aus Gründen des Verständnis vorerst mit den elementeweise definierten Ansatzfunktionen gearbeitet. Dabei ersetzen die elementweise definierten Ansatzfunktionen N_i^e und N_j^e im Interval des jeweiligen Elementes die zuvor für das gesamte Interval definierte Hütchenfunktion. Dabei wird auch die Matrix A^e wie auch Vektor $\vec{F}^e$ jeweils für die einzelnen Elemente definiert. Diese lassen sich folgenderweise berechnen:

$$A_{ij}^e = \int_a^b N_i^{e\prime}(x) N_j^{e\prime}(x)\, dx \tag{37}$$

$$\vec{F}_i^e = \int_a^b -f(x) N_i^e(x)\, dx \tag{38}$$

Dabei ist die Definitionsmenge des einzelnen Elementes im Interval $[a; b]$. A^e für ein 1 dimensionales Element (mit 2 Knoten) eine $2x2$ Matrix und $\vec{F}^e$ ein Vektor mit 2 Komponenten.

Betrachtung der Ableitung der Ansatzfunktion Durch Gleichungen (13) bis (15) wird uns die Ableitung der elementweise definierten Ansatzfunktionen durch die Ableitung der Ansatzfunktion des Referenzelement gegeben. Darauf wenden wir Integration durch Substitution an:

$$\int_a^b h(x)\,dx = \int_{\phi_e^{-1}(a)}^{\phi_e^{-1}(b)} h(\phi_e(\xi))\frac{\mathrm{d}\phi_e(\xi)}{\mathrm{d}} \tag{39}$$

Mit dem durch das Referenzelement gegebende Interval $[-1;1]$:

$$\int_a^b h(x)\,dx = \int_{-1}^{1} h(\phi_e(\xi))\frac{\mathrm{d}\phi_e(\xi)}{\mathrm{d}} \tag{40}$$

Damit können wir Gleichung (37) angehen.

$$A_{ij}^e = \int_a^b \frac{\mathrm{d}N_i^e(x)}{\mathrm{d}x}\frac{\mathrm{d}N_j^e(x)}{\mathrm{d}x}\,dx = \int_{-1}^{1} \frac{\mathrm{d}N_i^e(\phi_e(\xi))}{\mathrm{d}x}\frac{\mathrm{d}N_j^e(\phi_e(\xi))}{\mathrm{d}x}\frac{\mathrm{d}\phi_e(\xi)}{\mathrm{d}\xi}\,d\xi \tag{41}$$

Nun kann die Gleichung (15) für die Ableitung der Ansatzfunktionen eingesetzt werden.

$$A_{ij}^e = \int_{-1}^{1} \frac{\mathrm{d}N_i(\xi)}{\mathrm{d}\xi}\left(\frac{\mathrm{d}\phi_e(\xi)}{\mathrm{d}\xi}\right)^{-1}\frac{\mathrm{d}N_j(\xi)}{\mathrm{d}\xi}\left(\frac{\mathrm{d}\phi_e(\xi)}{\mathrm{d}\xi}\right)^{-1}\frac{\mathrm{d}\phi_e(\xi)}{\mathrm{d}\xi}\,d\xi \tag{42}$$

Dies wird ebenso auf Gleichung (38) angewandt.

$$\begin{aligned}\vec{F_i^e} &= \int_a^b -f(x)N_i^e(x)\,dx \\ &= \int_{-1}^{1} -f(\phi_e(\xi))N_i^e(\phi_e(\xi))\frac{\mathrm{d}\phi_e(\xi)}{\mathrm{d}\xi}\,d\xi \\ &= \int_{-1}^{1} -f(\phi_e(\xi))N_i(\xi)\frac{\mathrm{d}\phi_e(\xi)}{\mathrm{d}\xi}\,d\xi\end{aligned} \tag{43}$$

Der Vorteil dieser Schreibweise liegt darin, dass sie in dieser Form mit Hilfe der Gauß-

Quadratur effizient von Computern gelöst werden kann, da:

$$\int_{-1}^{1} F(\xi)\, d\xi = \sum_i F(\xi_i) w_i \tag{44}$$

Da wir exklusiv mit linearen Ansatzfunktionen arbeiten:

$$\int_{-1}^{1} F(\xi)\, d\xi = \sum_i F(\xi_i) w_i = 2F(0) \tag{45}$$

Korrelation zu der globalen Matrix und Vektor Aus den elementweise definierten Matrizen A^e und Vektoren $\vec{F}^e$ können wir die globale Matrix A und den globalen Vektor $\vec{F}$ konstruieren. In diesem Kontext bedeutet "global", dass diese Matrix sich auf das gesamte Intervall des Lösungs bezieht und mit diesen die gesamte diskrete Lösungsmenge $\vec{u}$ ermittelt werden kann. Dies kann über die elementweise definierte Lösung erreicht werden:

$$A^e \vec{u}^e = \vec{F}^e \tag{46}$$

Für ein 1-dimensionales Element:

$$\begin{bmatrix} A^e_{00} & A^e_{01} \\ A^e_{10} & A^e_{11} \end{bmatrix} \begin{pmatrix} u^e_0 \\ u^e_1 \end{pmatrix} = \begin{pmatrix} F^e_0 \\ F^e_1 \end{pmatrix} \tag{47}$$

Hierbei ist festzuhalten das der Knotenwert u^i_1 des i-ten Elements gleich dem Knotenwert u^{i+1}_0 des $(i+1)$-ten Elementes ist. Dasselbe gilt für F^i_1 und F^{i+1}_0

Konstruktion der globalen Matrix und Vektor Aus der obrigen Erkenntnis folgt, dass die globale Matrix A und der globale Vektor $\vec{F}$ aus einer Addition der elementweise definierten Matrizen A^e und Vektoren $\vec{F}^e$ an Stellen übereinstimmender Indizes i und j entstehen. So erreichen wir wieder das vorherige Linearen Gleichungssystem:

$$A\vec{u} = \vec{F}$$

4.7 Randbedingungen

Randbedingungen sind im Allgemeinen Umstände, die nur mit großem Aufwand oder gar nicht beeinflussbar sind oder sich aus der Problemstellung zwingend ergeben, und daher als gegebene Größen betrachtet werden müssen.
Im Bereich der Differentialgleichungen sind Randbedingungen konkrete Angaben zum Berechnen der Lösungsfunktion u auf einem Definitionsbereich D_u. Dazu werden die Werte der Funktion auf dem Rand von D_u vorgegeben. (vgl. wikipediaRandbedingung-Wikipedia)

4.7.1 Dirichlet-Randbedingung

Die Randbedingung erster Art (auch Dirichlet-Randbedingung genannt) ist ein Begriff aus der Mathematik und gehört zum Teilbereich der Theorie der Differentialgleichungen. Sie ist eine spezielle Randbedingung eines Randwertproblems, bei dem die Werte der gesuchten Funktion am Rand ihres Definitionsbereichs vorgegeben sind. (vgl. wikipedia-DirichletRandbedingungWikipedia)
Beispielsweise könnten die Dirichlet Randbedingungen einer gesuchten Lösungsfunktion $u(x)$, mit Definitionsbereich $D_u = [0, L]$, folgenderweise aussehen:

$$u(0) = 0$$

$$u(L) = 1$$

4.7.2 Neumann-Randbedingung

Eine Neumann-Randbedingung bezeichnet im Zusammenhang mit Differentialgleichungen Werte, die auf dem Rand des Definitionsbereichs für die Ableitung der Lösungsfunktion vorgegeben werden.(vgl. wikipediaNeumannRandbedingungWikipedia)

Beispielsweise könnten die Neumann-Randbedingung einer gesuchten Lösungsfunktion $u(x)$, mit Definitionsbereich $D_u = [0, L]$, folgenderweise aussehen:

$$u'(0) = 0$$

$$u'(L) = 0$$

4.7.3 Robin-Randbedingungen

Die Robin-Randbedingung legt eine Kombination des Funktionswertes und des Wertes ihrer Ableitung am Rand des Definitionsbereichs der Lösungsfunktion fest.(vgl. wikipediaRobinBoundary).

$$u(0) = 0$$

$$u'(L) = 0$$

4.8 Anwendung des FEM Verfahrens

Im Folgenden wird das FEM Verfahren für eine arbiträre PDE / DGL dargestellt:

- PDE, dessen Lösung zu finden ist, aufstellen
- Falls nötig, PDE in ihre Schwach Form umformulieren
- Randbedingungen definieren
- Ansatzfunktion wählen. Im Rahmen dieser Facharbeit wird sich exklusiv mit der linearen Ansatzfunktion auseinander gesetzt.
- Schwache Form diskretisieren
- Globale Matrix und Vektor aus elementweise definierten Matrizen und Vektoren konstruieren

- Finales lineares Gleichungsystem lösen
- stetige Lösungsfunktion mit diskreten Funktionswerten und Ansatzfunktion aufstellen

4.9 Poisson-Gleichung

Eine sehr bekannte DGL die viele Anwendungsbereiche in der Physik findet ist die sogenannte Poisson-Gleichung. Anhand der 1 dimensionalen ***Poisson-Gleichung*** wird die FEM illustriert, dabei wir eine Lösung im Interval $[0; 1]$ gesucht. Die ***Poisson-Gleichung*** kann in einer Dimension folgenderweise notiert werden:

$$\frac{\mathrm{d}^2 u}{\mathrm{d}x^2} = -f(x) \tag{48}$$

Schwache Form Da es sich um eine DGL zweiten Grades handelt, muss diese zuerst in die Schwache Form umformuliert werden, wie in Gleichung x-y gezeigt.

$$\int_0^1 \frac{\mathrm{d}^2 u}{\mathrm{d}x^2} v(x)\, dx = -\int_0^1 f(x) v(x)\, dx \tag{49}$$

Nun wird wieder partielle Integration angewendet.

$$\left[\frac{\mathrm{d}u(x)}{\mathrm{d}x} v(x)\right]_0^1 - \int_0^1 \frac{\mathrm{d}u}{\mathrm{d}x}\frac{\mathrm{d}v}{\mathrm{d}x}\, dx = -\int_0^1 f(x) v(x)\, dx \tag{50}$$

$$\frac{\mathrm{d}u(1)}{\mathrm{d}x} v(1) - \frac{\mathrm{d}u(0)}{\mathrm{d}x} v(0) - \int_0^1 \frac{\mathrm{d}u}{\mathrm{d}x}\frac{\mathrm{d}v}{\mathrm{d}x}\, dx = -\int_0^1 f(x) v(x)\, dx \tag{51}$$

Randbedingungen In diesem Beispiel werden Robin Randbedingungen benutzt. In diesem Fall gilt: $u(0) = 0$ und $u'(1) = 0$. Diese Randbedingungen lassen uns bereits die Schwache Form weiter vereinfachen. Da $u'(1) = 0$, kann diese folgendermaßen

umformuliert werden:

$$-\frac{\mathrm{d}u(0)}{\mathrm{d}x}v(0) - \int_0^1 \frac{\mathrm{d}u}{\mathrm{d}x}\frac{\mathrm{d}v}{\mathrm{d}x}\,dx = -\int_0^1 f(x)v(x)\,dx \tag{52}$$

Um diese Form also nochmal in einer bekannteren Weise zu illustrieren:

$$-u'(0)v(0) - \int_0^1 u'(x)v'(x)\,dx = -\int_0^1 f(x)v(x)\,dx \tag{53}$$

In dieser Form wurden alle Terme bezüglich der 2ten Ableitung u'' so gekürzt, dass nur noch die erste Ableitung im Fokus steht. Somit sind die Anforderungen an die FEM deutlich reduziert worden und machen das Problem leichter lösbar.

Diskretisierung In den nächsten Schritten werden wir die Schwache form der Poisson-Gleichung diskretisieren, dafür werden zuerst elementweise Matrizen K^e und Vektoren $\vec{F^e}$ berechnet. Zum Schluss wird daraus das lineare Gleichungssystem $K\vec{u} = \vec{F}$ aufgestellt.

Elementweise definierte Matrizen und Vektoren Aus Gleichung (37) ist uns Matrix K^e gegeben als:

$$K^e_{ij} = \int_a^b N_i^{e\prime}(x)N_i^{e\prime}(x)\,dx \tag{54}$$

Im Kontext der 1-dimensionalen gesuchten Lösungsfunktion ergibt sich:

$$K^e = l\begin{bmatrix} 1 & -1 \\ -1 & 1 \end{bmatrix} \tag{55}$$

Wobei l die Länge des Intervals eines einzelnen Elements darstellt. Unter der Annahme, dass alle n Elemente gleichlang sind: $l = \frac{1}{n}$. Der elementeweise definierte Vektor $\vec{F^e}$ ist gegeben durch Gleichung (38).

$$\vec{F_i^e} = \int_a^b -f(x)N_i^e(x)\,dx \tag{56}$$

Mit der Annahme $f(x) = f_{Konstant}$ im gesuchten Interval, ergibt sich:

$$\vec{F_i^e} = -f_{Konstant}\int_a^b N_i^e(x)\,dx \tag{57}$$

Dieser Vektor wird unter denselben Annahmen wie die Matrix evaluiert:

$$F^e = \frac{l}{2}\begin{pmatrix} f_{Konstant} \\ f_{Konstant} \end{pmatrix} \tag{58}$$

Aus diesen wird im späteren Program die globale Matrix K und Vektor $\vec{F}$ konstruiert:

$$K\vec{u} = \vec{F} \tag{59}$$

Das Lösen dieser Gleichung geschieht durch das Bilden der inversen Matrix K^{-1}, dies wird später ebenfalls von einem Computer-Program übernommen.

$$K^{-1}K\vec{u} = K^{-1}\vec{F} \tag{60}$$

$$\vec{u} = K^{-1}\vec{F} \tag{61}$$

Die Komponente des Vektors $\vec{u}$ sind die diskreten Funktionswerte; im späteren Program werden die Werte der Lösungsfunktion zwischen diesen diskreten Knoten durch die gewählte Ansatzfunktion errechnet.

4.10 Konvergenz in der FEM

Im zeitlichen Rahmen ist ein genauer mathematischer Beweis für die Konvergenz der Finite-Elemente-Methode nicht möglich, jedoch lässt sich graphisch aus Abbildung (3)

zumindest erahnen, dass die Finite-Elemente-Methode mit genügend Elementen der analytischen Lösung zumindest sehr nahe kommt.

$$\lim_{k\to\infty} u_{FEM}^{k} \approx u_{Analytisch} \tag{62}$$

5 Die Implementation

Nachdem die Theorie hinter der Finite Elemente Methode ausführlich erläutert wurde, beschäftigt sich dieser Abschnitt mit der Implementation einer solchen Methode. Um die Finite Elemente Methode auf einem Computer durchzuführen nehmen wir uns eine Programmiersprache zur Hilfe

5.1 Programmiersprache

Ein Programm ist eine Abfolge von Daten und Befehlen an einen Prozessor, um diese Daten in andere Daten umzuwandeln. Es wird in einer formal definierten Sprache verfasst, der Programmiersprache, (vgl. Henning und Vogelsang 2007, S. 10).
Eine Programmiersprache kann als eine Art formal definite Sprache gesehen werden, die es dem Mensch ermöglich Programme zu schreiben, welche von einem Computer ausgeführt werden.

5.2 Python

Python ist eine solche Programmiersprache. Die Implementation der Theorie dieser Facharbeit folgt in Python, aus Grunden der Leserlichkeit von Python-Code. Im Folgenden werden einige grundliegende Konzepte einer solchen Programmiersprache an Beispiel von Python dargestellt.

5.3 Grundlagen der Programmierung

Zuerst ist nur wichtig, dass ein Program in einer Datei gespeichert ist, welche nachher ausgeführt werden kann. Im Beispiel von Python haben diese Dateien die Dateiendung ".py", welche anzeigt, dass es sich um ein Python Program handelt.

Innerhalb einer solchen Datei kann sogennanter *Code* geschrieben werden. Als Programmcode werden die Anweisungen bezeichnet, die im Rahmen der Softwareentwicklung für ein bestimmtes Computerprogramm entstehen (vgl. wikipediaProgrammcode-Wikipedia). Sehr vereinfach kann sich Code als eine Art Text vorgestellt werden, welcher dem Computer sagt, was dieser zu machen hat. Im Folgenden werden kleine Abschnitte von Python Code gezeigt, welche fundamentale Grundbausteine, die verwendet werden um komplexe Programme zu schreiben, darstellen.

5.3.1 Kommentare

Bevor wir uns mit der eigentlich Syntax von Python beschäftigen, wird das Benutzen von sogenannten ***Kommentaren*** eingeführt. Diese Kommentare sind Textzeilen, welche beim Ausführen eines Programmes ignoriert werden. Sie werden von Programmieren benutzt, um mehr Informationen über Code-Blöcke zu geben.

```
#Dies ist ein Python Kommentar
```

5.3.2 Variablen

Variablen fungieren als abstrakter benannter Behälter für einen Wert. Diese werden oft benutzt um Werte zu speichern, die mehrmals im Program benutzt werden, als auch um den Code lesbarer zu machen.

```
#Die folgende Zeile Code erstellt eine Variable "a" und gibt ihr den
↪  Wert "10"
a = 10
#Der Wert von Variablen kann geäandert werden
a = a + 1#a besitzt danach den Wert 11
```

5.3.3 Funktionen

Eine ***Funktion*** ist ein benannter Block Code, welcher mit Hilfe des Funktionsnamens an beliebiger Stelle innerhalb eines Programmes aufgerufen und damit ausgeführt werden kann. Wichtig ist dabei zu beachten, dass eine Funktion erst *definiert* werden muss, bevor diese ausgeführt werden kann. Zusätlich können Funktionen optional mit ***Parametern*** definiert werden, dies sagt dem Program, dass eine solche Funktion beim Aufrufen gewisse *Argumente* gegeben werden müssen. Hinzu kann eine Funktion einen ***Rückgabewert*** liefern, dies ist das Ergebnis, das eine Funktion nach ihrer Ausführung an die aufrufenden Stelle im Programm zurückliefert.

```
1  #Der folgende Code deklariert eine neue Funktion namens "funktion1"
2  def funktion1():
3      ... #<= An diese Stelle kommt der Code Block, welche beim aufrufen
       ↪  der Funktion ausgeführt werden soll
4
5  funktion1() #<= dies führt die Funktion "funktion1" aus
6
7  def add(a, b):#Diese Funktion ist mit den Parametern "a" und "b"
   ↪  definiert
8      return a + b
9
10 c = add(2, 10)#Diese Funktion wird mit den Argumenten "2" und "10"
   ↪  aufgerufen und der Rückgabewert "12" wird in der Variable "c"
   ↪  gespeichert
11 d = add(c, 8)#Eine Funktion kann auch eine Variable als Argument
   ↪  erhalten
```

Python besitzt bereits Funktionen, die wir nicht selber definieren müssen, diese heißen *built-in functions*. Eine solcher built-in funktion ist *print*, sie druckt Text oder andere Infor-

mationen aus.

```
print("In Variable d liegt nun der Wert:", d)#Dies druckt den Text "In
↪ Variable d liegt nun der Wert: 20" aus
```

5.3.4 Datentypen

Programme arbeiten mit verschiedensten Daten, diese werden in verschiedene Datentypen unterschieden. In Python kann u. a. mit folgenden Datentypen gearbeitet werden:

- Integer (Ganzzahlen)
- Float (Zahlen mit Nachkommastellen)
- String (Text)
- List (um eine Reihe an Werten darzustellen)

Verschiedene Datentypen unterscheiden sich in den Werten die sie speichern, wie auch den Operationen die auf sie anwendbar sind.

```
a = 10# Ein Integer "10" wird in der Variable a gespeichert
a = a + 5#Ein Weiterer Integer wird addiert, a hat nun den Wert "15"

b = 0.5#Ein Float "0.5" wird in der Variable b gespeichert
b = b + 0.1#Ein weiterer Float wird addiert, b hat nun den Wert "0.6"
b = b + 5#Einige Datentypen wie Floats und Integer können auch
↪ operationen teilen, dabei wird der Integer von Python in ein Float
↪ umgewandelt, ohne dass wir etwas machen müssen. "b" hat den Wert
↪ "5.6"

c = "Hallo Welt"
```

```
d = [5, 12, 7]#Eine Liste von Integern, in Python können die einzelnen Elemente einer Liste verschiedene Datentypen besitzen, obwohl dies in guten Code nicht nötig ist.
print(d[1])#d[i] ist das i-te Element der Liste, wobei nicht von 1 sonder 0 gezählt wird. In diesem Fall ist d[1] das 2-te Element der Liste "d" und druckt "12" aus
```

5.3.5 Schleifen

Schleifen erlauben gewisse Codeblöcke wiederholt auszuführen. In Python gibt es fundamental zwei Arten von Schleifen.

For-Schleife Eine For-Schleife lässt einen Codeblock eine vordefinierte Anzahl wiederholen.

```
for i in range(10):#Wiederholt den Codeblock 10 mal, dabei zeigt i die aktuelle Iteration an
    #Der zu wiederholende Codeblock ist mit einem Tab einzurücken
    print("Hallo Welt")
```

While-Schleife Eine While-Schleife führt einen Codeblock wiederholt aus, bis eine gewisse Aussage nicht mehr wahr ist.

```
a = 0
while (a<10):#Wiederholt den Codeblock bis a >= 10 ist
    print("Hallo Welt")
    a += 1#erhöht a um 1
```

5.3.6 OOP

Python ist eine sogenannte ***Objektorientierte Programmiersprache***. Objektorientierte Programmierung (OOP) ist ein sehr ausführliches Thema und wir durch den Rahmen der Facharbeit nur kurz betrachtet. Grundlegend handelt OOP von der Idee, Code in sogenannte ***Objekte*** zu abstrahieren, diese beinhalten sowohl Daten wie auch Funktionen. Dies ermöglicht es viele Probleme deutlich intuitiver anzugehen.

```
#oop

class Customer:#eine Klasse ist wie ein Bauplan mit dem Objekte erstellt werden
    def __init__(self, name):#die __init__ Funktion ist der sog. Konstruktor, er wird aufgerufen, wenn ein neues Objekt dieser Klasse erstellt werden soll, dabei muss das Objekt die Parameter dieser Methode(Funktion) angeben
        self.name = name# "self" ist hier wie eine Art Platzhalter für das jeweilige Objekt, also hat jedes Objekt dieser Klasse eine Variable "name"
        self.money = 0#und eine Variable "money"

    def deposit_money(self, amount):
        self.money += amount

    def send_money(self, other_customer, amount):
        if amount > self.money:
            raise Exception("Dieser Kunde hat nicht so viel Geld!")
        self.money -= amount
```

```
15          other_customer.money += amount#other_customer ist ein weiteres
            ↪  Objekt dieser Klasse und stellt einen anderen Kunden der,
            ↪  dem Geld überwiesen werden soll
16
17 #karl ist ein Objekt, dieses Objekt ist eine Instanz der Klasse
   ↪  "Customer"
18 #dies Bedeutet, dass das Objekt "karl" mit dem Bauplan der Klasse
   ↪  "Customer" aufgebaut wurde und dessen Struktur folgt
19 karl = Customer("Karl")
20 karl.deposit_money(1000)#das Objekt verfügt über Funktionen
21 print(karl.money)#wie auch über Variablen (Daten)
22
23 susy = Customer("Susy")
24 susy.deposit_money(500)
25
26 print("Vor der Transaktion", karl.money, susy.money)
27 karl.send_money(susy, 500)
28 print("Nach der Transaktion", karl.money, susy.money)
29
```

5.3.7 Bibliotheken

Python stellt sogennante ***Module (engl. Modules*** und ***Packete (engl. Packages)*** zur Verfügung, diese Erweitern Python's Funktionalität.

Module Ein Modul ist eine Datei, die Python-Definitionen und Anweisungen enthält. Der Dateiname ist der Modulname mit der angehängten Endung .py. . (vgl. pythonModules)

Package In Python sind Packages eine Methode, um den Namensraum von Modulen mithilfe von „punktierten Modulnamen“ zu strukturieren.(vgl. pythonPackages) Diese werden auch oft Libaries (auf deutsch Bibliotheken) genannt.

Meist beschäftigt sich ein Package mit einer gewissen Funktionalität. Einige bekannte Python Packages:

- Numpy
  - Eine Bilbiothek die für mathematische Operationen, wie das Rechnen mit Vektoren oder Matrizen, verwendet wird
- Matplotlib
  - Eine Bilbiothek die verwendet wird um Daten graphisch darzustellen

Um eine Bilbiothek in Python zu benutzen, muss diese zuerst ***importiert*** werden. Dies geschieht meist am Anfangs eines Programmes:

```
import numpy #importiert die Bilbiothek "NumPy"
import numpy as np #import die Bibliothek, jedoch unter dem Namen
↪  "np" (dies erleichtert die Schreibarbeit)

#erstellt eine Matrix "M"
M = np.matrix([
    [1, 2, 3],
    [4, 5, 6]
])
#erstellt einen Vektor "x"
x = np.array([
    1, 2, 3
])
```

```
    #c speichert den resulierenden Vektor, wenn man die Matrix M mit dem
    ↪  Vektor x multipliziert
    c = np.dot(M, x)

    print(c)#gibt das Ergebnis aus
```

5.3.8 Pip

Pip ist Python's eingebauter Package Manager, der es ermöglich Packages herunterzuladen, welche von anderen Leuten geschrieben und veröffentlicht wurden. Pip ermöglicht über die Powershell wie die Commandline Shell das installieren von Packages folgender Maßen:

```
pip install package-name
```

So kann Beispielsweise NumPy folgender weise installiert werden:

```
pip install numpy
```

Hinzu kommt es in größeren Projekten dazu, dass viele Packages mit speziellen Versionen benutzt werden. Um deren Installation zu erleichertn wird meist eine ***"requirements.txt"*** Datei erstellt. Diese Datei enthält alle Informationen die ***Pip*** benötigt um die Packete zu installieren, dazu kann folgender Befehl ausgeführt werden

```
pip install -r requirements.txt
```

5.3.9 Jupyter Notebooks

Jupyter Notebooks sind eine besondere Art von Python Dateien. Diese haben die Dateiendung „.ipynb". Anstelle ein gesamtes Program auf einmal auszuführen, ermöglichen

diese, kleine Code-Snippets einzeln auszuführen und beispielsweise graphische Ergebnisse direkt in der Datei anzeigen zu lassen.

5.4 Poisson Gleichung

Unter folgendem Link ist der Code für das Program zu finden:
https://github.com/NightlyProgrammer/FacharbeitPhysikFEM/blob/main/src/poission_equation.ipynb
Abbildung 3 wurde mit Hilfe dieses Programmes erstellt. Dabei würde für $f_{Konstant}$ ein Wert von 2 gewählt. Viele Variablen des Programmes wie die Menge an finiten Elementen *num_elements* kann verändert werden. Dabei sei zu beachten, dass die analytische Lösung nur für gewisse Bedingung gelöst wurde. Dies bedeutet, dass die analytische Lösung unter Umständen von der FEM Lösung abweicht, dies liegt allerdings nicht daran, dass die FEM Lösung falsch ist, sondern die analytische Lösung nicht für diese Umstände ausgelegt war. Folgende Variablen lassen sich verändern:

- *num_elements*
  - Diese Variable gibt die Menge an diskreten Elementen an.
- *f_constant*
  - Diese Variable gibt den konstanten Wert an, den die Funktion $f(x)$ über das Interval annimmt.
- *fixed_values*
  - Dieses Array gibt die zwei Werte für die Dirichlet-Randbedingung an. Dabei ist der erste Wert *fixed_values[0]* $u(0)$ und der zweite Wert *fixed_values[1]* $u(L)$ zuzuschreiben.

6 Fazit

Das Ziel dieser Facharbeit wurde im Rahmen der Erläuterung der Finite-Elemente-Methode und derren beispielsweisen Anwendung erreicht. Zusammenfassend kann festgehalten werden, dass die Methodik der Anwendung der FEM erläutert wurde und kurz anhand der Poisson-Gleichung illustriert wurde. Des weiteren wurden Programme hinterlegt, welche diese Methodik anschaulicher machen sollten. Rückblickend kann jedoch gesagt werden, dass die Komplexität des Thema durch meine ursprünglich sehr weit gefasste Zielsetzung unterschätzt wurde. So war eigentlich angedacht, das FEM Verfahren ebenfalls auf die PDG der Linearen Elastizität anzuwenden, dies war im gegebenden Zeitrahmen nicht realisierbar.

6.1 Mögliche weiterführende Themen und Bereiche

Durch die Limitationen einer Facharbeit musste ein sehr komplexes Thema deutlich kurz gehalten werden. Dies führt dazu, dass viele wichtige Aspekte deutlich weniger im Fokus lagen, als es ihrer Natur eigentlich gegeben ist. Diese Aspekte können in einige Teilbereiche unterteilt werden und könnten sich unter anderem für weitere Facharbeiten / wiss. Arbeiten eignen. Im Folgenden werden einige dieser Bereiche erläutert, wobei der Fokus größenteils auf der FEM bestehen bleibt:

- Weitere DGL mit der FEM lösen
  - Die offensichtlichste Weiterführung dieses Werkes besteht darin, die hier vorgestellte FEM auf weitere für die Physik relevante DGL anzuwenden. Einige Namenswerte Gleichungen sind dabei:
    - Lineare Elastizität
    - Wärmeleitungsgleichung

  - Hinzu werden in vielen FEA verschiedene Eigenschaften gleichzeitig untersucht. Es wäre möglich zu untersuchen, wie sich verschiedene DGL / PDG vereinen lassen, um beispielsweise sowohl die Verformung wie die Wärmeverteilung eines Bauteils zu modelieren.

- Tiefer in die Ansatzfunktionen eintauchen
  - Im Rahmen der Facharbeit wurden die Funktionen, welche Werte zwischen den Knoten interpolieren, nur sehr kurz untersucht. Weitere Untersuchungen könnten zu den verschiedenen Anwendungszwecken verschiedener Funktionen getätigt werden. Wie auch diese genauer miteinander zu vergleichen.

- Auf den mathematischen Beweis der Konvergenz der Finite-Elemente-Methode eingehen.
  - Im Rahmen dieser Facharbeit würde nur graphisch an Abbildung (3) gezeigt, dass $u_{FEM}^{k} = u_{Analytisch}$ für $k \to \infty$, wobei k die Menge an Elementen darstellt.

Literatur-/Quellenverzeichnis

[pythonModules] ***6. Modules — docs.python.org***. https://docs.python.org/3/tutorial/modules.html. – [Accessed 03-03-2026]

[pythonPackages] ***6. Modules — docs.python.org***. https://docs.python.org/3/tutorial/modules.html#packages. – [Accessed 03-03-2026]

[wikipediaAnalysisWikipedia] ***Analysis – Wikipedia — de.wikipedia.org***. https://de.wikipedia.org/wiki/{A}nalysis. – [Accessed 27-02-2026]

[wikipediaDirichletRandbedingungWikipedia] ***Dirichlet-Randbedingung – Wikipedia — de.wikipedia.org***. https://de.wikipedia.org/wiki/{D}irichlet-{R}andbedingung. – [Accessed 01-03-2026]

[wikipediaDiskretisierungWikipedia] ***Diskretisierung – Wikipedia — de.wikipedia.org***. https://de.wikipedia.org/wiki/{D}iskretisierung. – [Accessed 02-03-2026]

[wikipediaGalerkinMethodeWikipedia] ***Galerkin-Methode – Wikipedia — de.wikipedia.org***. https://de.wikipedia.org/wiki/{G}alerkin-{M}ethode. – [Accessed 02-03-2026]

[wikipediaGauQuadraturWikipedia] ***Gauß-Quadratur – Wikipedia — de.wikipedia.org***. https://de.wikipedia.org/wiki/{G}au%{C}3%9{F}-{Q}uadratur. – [Accessed 03-03-2026]

[wikipediaIntegrationSubstitution] ***Integration durch Substitution – Wikipedia — de.wikipedia.org***.

https://de.wikipedia.org/wiki/{I}ntegration_durch_{S}ubstitution. – [Accessed 03-03-2026]

[wikipediaLineareAlgebra] ***Lineare Algebra – Wikipedia — de.wikipedia.org***. https://de.wikipedia.org/wiki/{L}ineare_{A}lgebra. – [Accessed 08-02-2026]

[wikipediaMatrixMathematik] ***Matrix (Mathematik) – Wikipedia — de.wikipedia.org***. https://de.wikipedia.org/wiki/{M}atrix_({M}athematik). – [Accessed 08-02-2026]

[wikipediaMatrizenmultiplikationWikipedia] ***Matrizenmultiplikation – Wikipedia — de.wikipedia.org***. https://de.wikipedia.org/wiki/{M}atrizenmultiplikation. – [Accessed 28-02-2026]

[wikipediaNeumannRandbedingungWikipedia] ***Neumann-Randbedingung – Wikipedia — de.wikipedia.org***. https://de.wikipedia.org/wiki/{N}eumann-{R}andbedingung. – [Accessed 01-03-2026]

[wikipediaPartielleDifferentialgleichung] ***Partielle Differentialgleichung – Wikipedia — de.wikipedia.org***. https://de.wikipedia.org/wiki/{P}artielle_{D}ifferentialgleichung. – [Accessed 27-02-2026]

[wikipediaPartielleIntegration] ***Partielle Integration – Wikipedia — de.wikipedia.org***. https://de.wikipedia.org/wiki/{P}artielle_{I}ntegration. – [Accessed 01-03-2026]

[wikipediaProgrammcodeWikipedia] ***Programmcode – Wikipedia — de.wikipedia.org***. https://de.wikipedia.org/wiki/{P}rogrammcode. – [Accessed 08-02-2026]

[wikipediaRandbedingungWikipedia] ***Randbedingung – Wikipedia — de.wikipedia.org***. https://de.wikipedia.org/wiki/{R}andbedingung. – [Accessed 01-03-2026]

[wikipediaRobinBoundary] ***Robin boundary condition - Wikipedia — en.wikipedia.org***. https://en.wikipedia.org/wiki/{R}obin_boundary_condition. – [Accessed 01-03-2026]

[wikipediaWeakFormulation] ***Weak formulation - Wikipedia — en.wikipedia.org***. https://en.wikipedia.org/wiki/{W}eak_formulation. – [Accessed 28-02-2026]

[wiasberlin] ***wias-berlin.de***. https://www.wias-berlin.de/people/john/{L}{E}{H}{R}{E}/{N}{U}{M}_{P}{D}{E}/{S}{S}09/num_pde_intro_04.pdf. – [Accessed 02-03-2026]

[wiki:Einheitsmatrix 2026] ***Einheitsmatrix — Wikipedia, The Free Encyclopedia***. http://de.wikipedia.org/w/index.php?title=Einheitsmatrix&oldid=262592522. 2026. – [Online; accessed 24-February-2026]

[Heinzle] HEINZLE, J. M.: ***Gewöhnliche Differentialgleichungen***. https://gravity.univie.ac.at/fileadmin/user_upload/i_gravity_physics/material/teaching/m1/M1ODEs.pdf. – [Accessed 07-02-2026]

[Henning und Vogelsang 2007] HENNING, Peter A. ; VOGELSANG, Holger: Handbuch Programmiersprachen. In: ***Softwareentwicklung zum Lernen und***

Nachschlagen:[ABAP, Ada, C, C#, C++, Delphi, Eiffel, FORTRAN, Java, JavaScript, LISP, Maple, Mathematica, MATLAB, Perl, PHP, PROLOG, Python, Ruby, Smalltalk, SQL, Tcl/Tk, Visual Basic .NET]. Hanser, München (2007)

[Oluwole 2011] OLUWOLE, Oluleke: The Weak Formulation. In: ***Finite Element Modeling for Materials Engineers Using MATLAB®*** (2011), S. 3–12

[Yaniv] YANIV, Ariel: ***Finite Element Method Explained: How to Solve the 1D Poisson Equation—Part I — ariel.yaniv***. `https://medium.com/@ariel.yaniv/finite-element-method-explained-how-to-solve-the-1d-poisson-equation-part-i-206b5b` – [Accessed 02-03-2026]

Weitere Quellen

Diese Quellen wurden nicht als primäre Literatur verwendet, sondern dienten dazu, ein erstes intuitives Verständnis aufzubauen. Viele dieser Quellen liegen im Format eines Videos vor.

- https://youtu.be/GHjopp47vvQ?si=r28FMCUR8JQwsSHR
- https://youtu.be/4SkxQIejVfs?si=56zwOjvnTmS-NXYj
- https://youtu.be/zQJkve_hnHk?si=81qlsep7mVpNDevt
- https://youtu.be/Z-FnP2myvKw?si=Rn7DbF0QG9_ThRQ5
- https://youtu.be/1wSE6iQiScg?si=yvc62KqoHD03CyFj
- https://youtu.be/VjMFz63RmSQ?si=fYXmN6f9TcTUZwNy
- https://youtu.be/xZpESocdvn4?si=W0Ex3SC427yEg4kn

Nutzung von künstlicher Intelligenz

Im Rahmen dieser Facharbeit wurde KI für folgende Bereiche genutzt:

- Rechtschreib- und Sprachprüfung.
- LaTeX:
  - Da LaTeX für mich ein neues Wissensgebiet darstellte, habe ich mich bei der Erstellung von Formeln und der Dokumentenstruktur teilweise der Unterstützung einer KI bedient.
- Erstellung der Abbildung 2:
  - Da diese lediglich grafisch eine Korrelation darstellen sollte und keine realen Daten berechnet wurden, wurde der Programmcode zur Erzeugung der Grafik KI-generiert.
- Verifizierung der analytischen Lösung:
  - Da der Fokus auf der numerischen Lösung lag, wurde die analytische Lösung lediglich zu Vergleichszwecken mithilfe von KI verifiziert.

Anhang

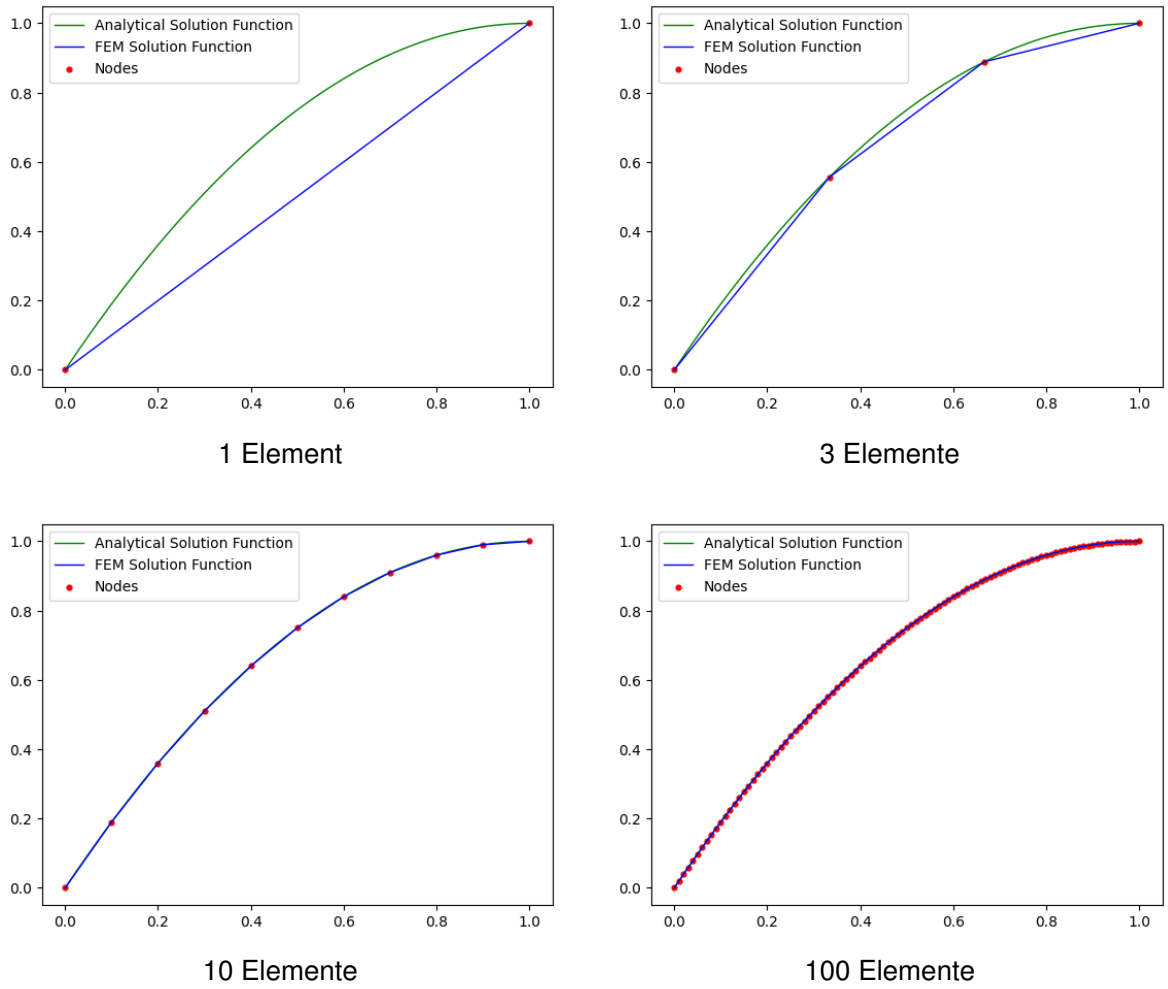

Abbildung 3: Vergleich der FEM-Lösung mit der analytischen Lösung. Die roten Punkte repräsentieren die Knoten (Nodes), an denen die diskreten Werte berechnet wurden. Der restliche Graph ergibt sich durch Interpolation mittels der Ansatzfunktionen.

Schlusserklärung

Hiermit erkläre ich, dass ich die vorliegende Arbeit selbstständig und ohne fremde Hilfe verfasst und keine anderen als die im Literaturverzeichnis angegebenen Hilfsmittel verwendet habe. Insbesondere versichere ich, dass ich alle wörtlichen und sinngemäßen Übernahmen aus anderen Werken als solche kenntlich gemacht habe

Ort, Datum

Unterschrift (Vorname Nachname)